

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Redes

Semestre II – 2026



Excelencia que trasciende

DEL VALLE
GRUPO EDUCATIVO

Proyecto 1

Uso de un protocolo existente

Integrantes:

Nájera Marakovits, Ingrid Nina Alessandra – 231088

Resumen

El proyecto implementa un chatbot de consola que utiliza la API REST de Gemini y Model Context Protocol. El anfitrión mantiene el contexto de la conversación y coordina servidores para Filesystem, Git y operaciones de red. El cliente MCP, el servidor personalizado y los transportes local y remoto se implementaron manualmente con JSON-RPC 2.0.

El servidor de operaciones de red ofrece cuatro herramientas de diagnóstico de solo lectura. La versión local utiliza entrada y salida estándar. La versión remota utiliza MCP Streamable HTTP sobre HTTPS y se encuentra desplegada en Render. La comunicación remota fue capturada con Wireshark para analizar la sincronización, las solicitudes, las respuestas y las capas de red.

1 Objetivos

- Conectar un chatbot con Gemini mediante su API REST.
- Mantener el contexto durante una sesión de conversación.
- Integrar los servidores oficiales Filesystem y Git.
- Desarrollar manualmente un servidor MCP para operaciones de red.
- Publicar el servidor mediante HTTPS en Render.
- Capturar y clasificar el tráfico MCP con Wireshark.
- Validar la solución con pruebas automatizadas y ejecuciones reales.

2 Arquitectura y funcionamiento

El archivo `src/chatbot.py` funciona como anfitrión. La comunicación con Gemini se realiza mediante solicitudes HTTP y la comunicación con los servidores MCP se realiza desde `src/mcp_client.py`. El servidor personalizado se encuentra en `src/network_server.py` y el transporte remoto en `src/remote_server.py`.

Componente	Transporte	Función
Gemini	HTTPS	Interpretar la solicitud y redactar la respuesta
Filesystem	Entrada y salida estándar	Operaciones controladas sobre archivos
Git	Entrada y salida estándar	Estado, preparación, commits e historial
Network local	Entrada y salida estándar	Diagnósticos de red locales
Network remoto	HTTPS	Diagnósticos desde el servicio en Render

Tabla 1: Componentes principales de la solución

El flujo comienza cuando el usuario escribe una solicitud. El anfitrión guarda el mensaje en el historial y envía a Gemini solamente las herramientas relacionadas. Si Gemini solicita una función, la consola muestra el nombre y los argumentos. Luego de la confirmación, el cliente envía `tools/call` al servidor. El resultado vuelve a Gemini para producir la respuesta final.

El ciclo MCP utiliza `initialize`, `notifications/initialized`, `tools/list` y `tools/call`. Las solicitudes incluyen un identificador y las respuestas conservan el mismo valor. Las notificaciones no tienen identificador y no producen una respuesta JSON-RPC.

3 Integración con Gemini

La conexión utiliza el endpoint `generateContent` de Gemini. El historial conserva los mensajes del usuario, las respuestas del modelo y los resultados de herramientas. Una prueba real solicitó información sobre Alan Turing y luego preguntó el año de nacimiento. El modelo respondió 1912 al utilizar el contexto anterior.

El chatbot registra las interacciones en `logs/mcp.jsonl`. Cada entrada contiene fecha, servidor, dirección y mensaje. Los secretos se cargan desde `.env`, archivo excluido de Git. También se agregaron reintentos para errores temporales de Gemini y selección contextual de herramientas para reducir solicitudes innecesarias.

4 Servidores oficiales

El servidor Filesystem se limita al directorio del proyecto. El servidor Git trabaja sobre el repositorio actual. La demostración creó `docs/mcp-demo/README.md`, agregó únicamente ese archivo al área de preparación y generó un commit desde el servidor MCP Git. El estado final confirmó que ambas integraciones funcionaron correctamente.

5 Servidor MCP de operaciones de red

El servidor se identifica como `uvg-network-operations` y utiliza la versión MCP 2025-11-25. Su propósito es apoyar tareas de una mesa de ayuda mediante verificaciones pequeñas y de solo lectura.

Herramienta	Entrada	Resultado
<code>resolve_dns</code>	hostname e ip version opcional	Direcciones IP únicas
<code>check_tcp_connection</code>	host, puerto y tiempo opcional	Estado y tiempo de conexión
<code>analyze_cidr</code>	Red en formato CIDR	Red, máscara, broadcast y rango útil
<code>get_local_network_info</code>	Sin parámetros	Nombre del equipo y direcciones locales

Tabla 2: Herramientas del servidor personalizado

Los argumentos se validan mediante esquemas JSON. Una entrada inválida devuelve un resultado con `isError` en verdadero sin detener el servidor. La ejecución de `analyze_cidr` para `192.168.100.70/26` produjo la red `192.168.100.64/26`, el broadcast `192.168.100.127` y 62 direcciones utilizables.

6 Despliegue remoto

La misma lógica se ejecuta en un contenedor Docker publicado en Render. El servicio utiliza un usuario sin privilegios, recibe el puerto mediante la variable `PORT` y requiere autenticación Bearer para MCP. El token no se almacena en el repositorio.

Endpoint	Método	Autenticación	Resultado
<code>/health</code>	GET	No	Estado, nombre y versión
<code>/mcp</code>	POST	Bearer	Solicitudes MCP mediante JSON
<code>/mcp</code>	GET	Bearer	Código 405
<code>/mcp</code>	DELETE	Bearer	Código 405

Tabla 3: Endpoints del servicio remoto

El health check público respondió HTTP 200. El cliente remoto negoció la versión MCP, descubrió cuatro herramientas y ejecutó correctamente `analyze_cidr` y `resolve_dns`. La dirección del servicio es <https://uvg-network-operations.onrender.com>.

7 Captura con Wireshark

La comunicación utiliza TLS 1.3. Para observar JSON-RPC se guardaron temporalmente las claves de la sesión TLS del proceso de prueba. Wireshark utilizó esas claves para

descifrar únicamente la conexión con Render. La captura se limitó al puerto 443 y produjo 91 paquetes. El archivo de captura y las claves permanecen fuera de Git.

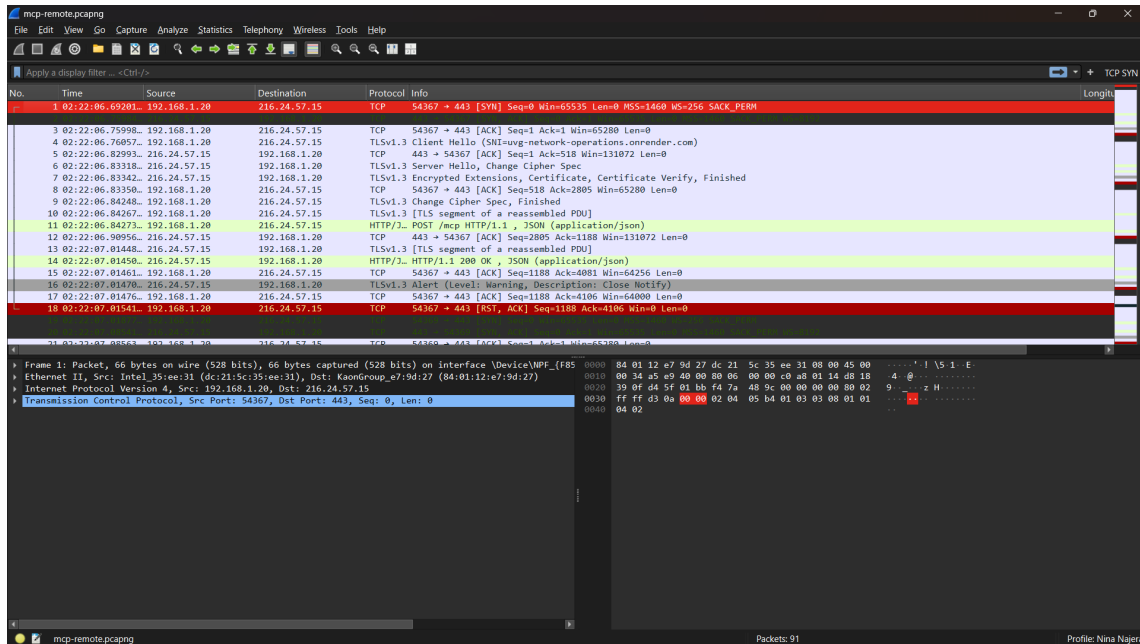


Figura 1: Establecimiento TCP y negociación TLS 1.3

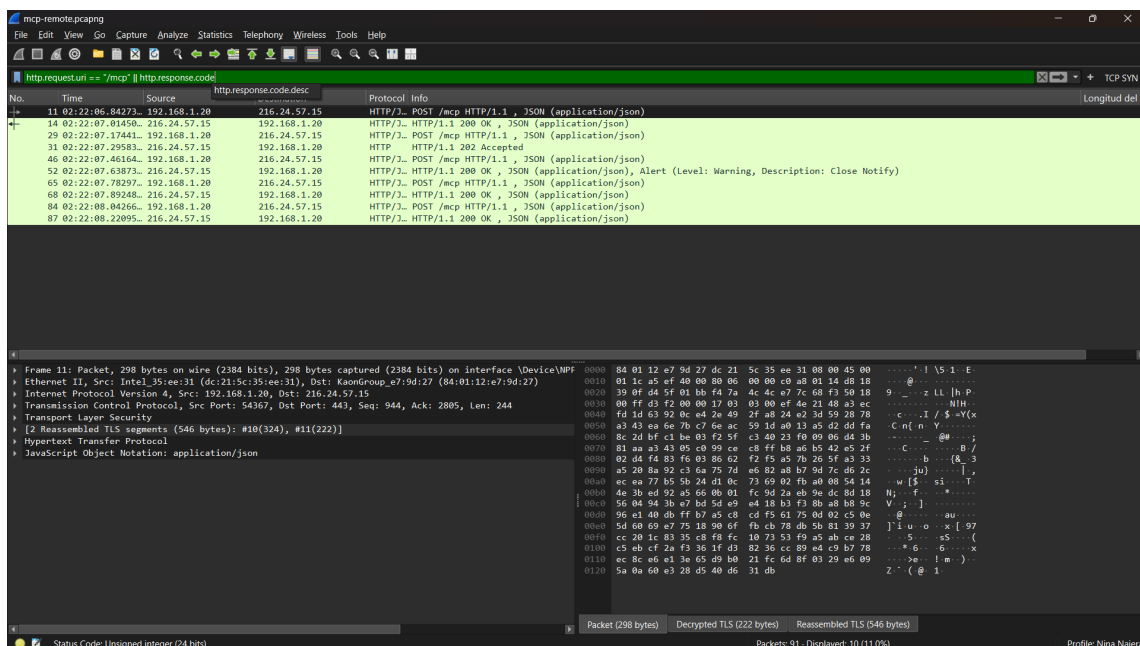


Figura 2: Solicitudes y respuestas HTTP del ciclo MCP

7.1 Análisis por capas

En la capa de enlace, Ethernet II transporta el datagrama dentro de la red local. En la capa de red, IPv4 identifica al cliente y al servidor remoto. En la capa de transporte,

TCP utiliza el puerto 443 y garantiza entrega ordenada. TLS 1.3 autentica el dominio y cifra HTTP. En la capa de aplicación, HTTP transporta objetos JSON-RPC y MCP define el significado de los métodos.

8 Clasificación de mensajes

Trama	Dirección	Mensaje	Clasificación
11	Cliente a servidor	initialize, id 1	Solicitud de sincronización
14	Servidor a cliente	result, id 1	Respuesta de sincronización
29	Cliente a servidor	notifications/ initialized	Notificación de sincronización
31	Servidor a cliente	HTTP 202	Confirmación sin cuerpo JSON
46	Cliente a servidor	tools/list, id 2	Solicitud de herramientas
52	Servidor a cliente	result.tools, id 2	Respuesta de herramientas
65	Cliente a servidor	tools/call, id 3	Solicitud de ejecución
68	Servidor a cliente	result, id 3	Respuesta de ejecución
84	Cliente a servidor	tools/call, id 4	Segunda solicitud
87	Servidor a cliente	result, id 4	Segunda respuesta

Tabla 4: Clasificación del intercambio MCP

8.1 Sincronización

La solicitud initialize propone la versión del protocolo y describe el cliente. La respuesta acepta la versión y devuelve información del servidor. La notificación notifications/initialized indica que el cliente terminó la inicialización.

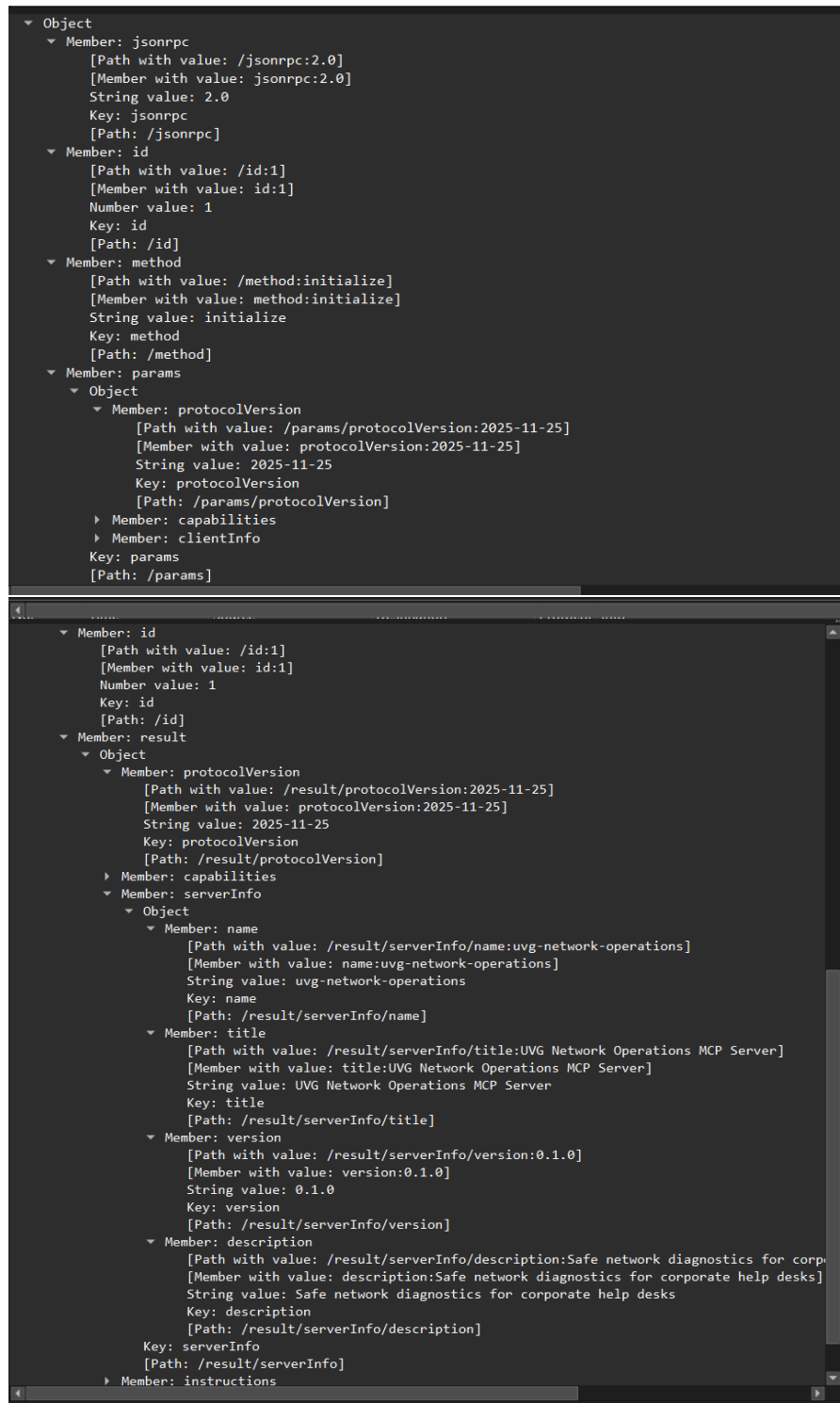


Figura 3: Solicitud y respuesta de inicialización

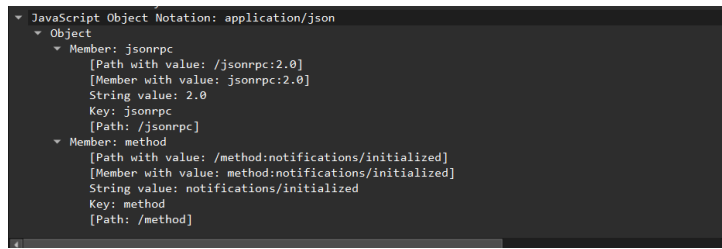


Figura 4: Notificación de inicialización sin identificador

8.2 Descubrimiento y ejecución

La solicitud `tools/list` obtiene los nombres, descripciones y esquemas de las cuatro herramientas. La solicitud `tools/call` ejecuta una herramienta con argumentos validados y conserva el identificador en la respuesta.

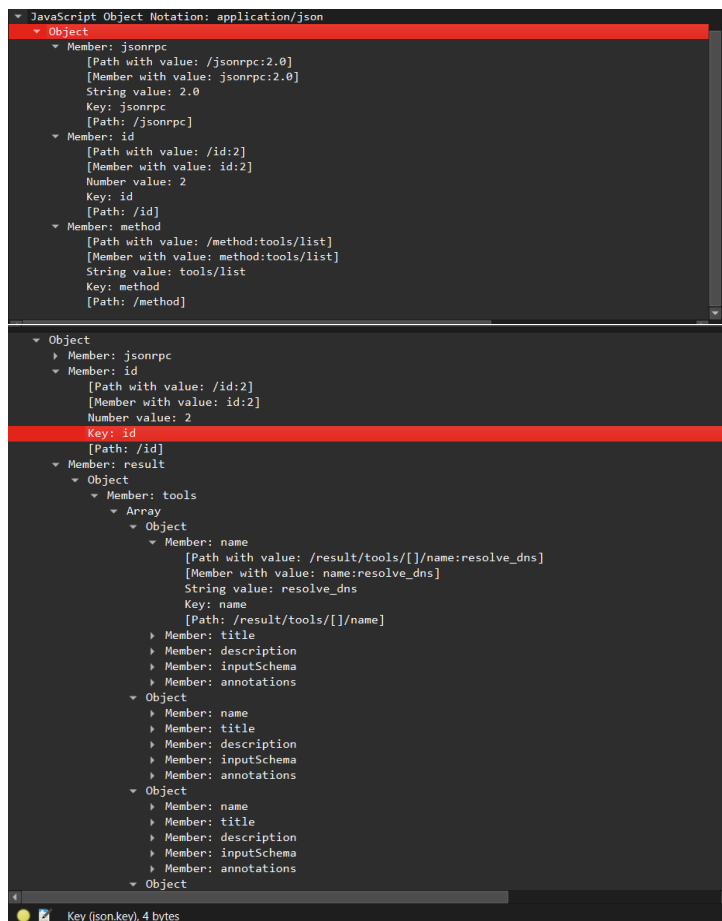


Figura 5: Solicitud y respuesta del descubrimiento de herramientas

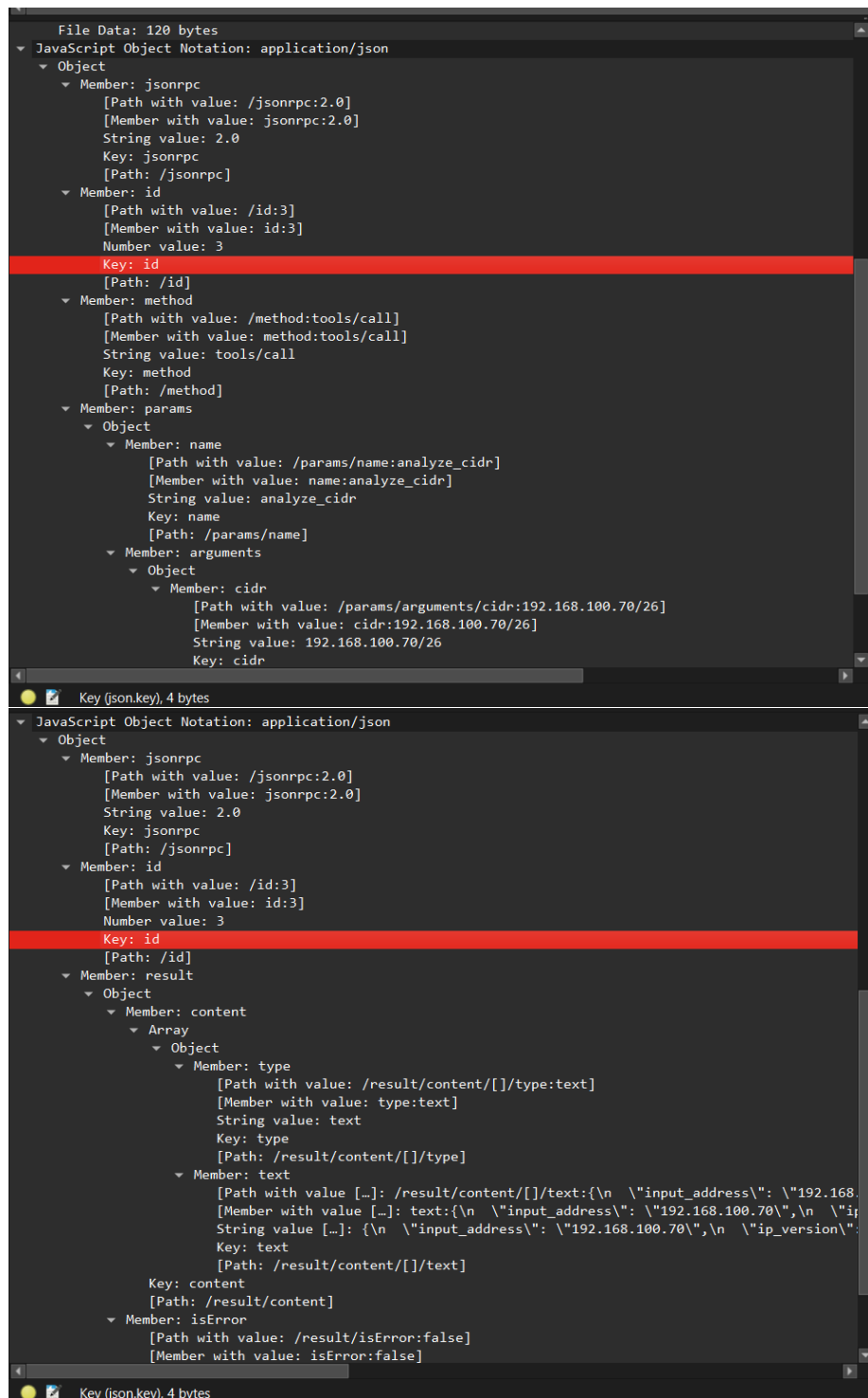


Figura 6: Solicitud y respuesta de la ejecución de una herramienta

9 Pruebas y resultados

La suite automatizada finalizó con 16 pruebas aprobadas. Se validaron los esquemas del servidor, el transporte local, el transporte HTTP, la autenticación, la versión del protocolo, el contexto de Gemini, los reintentos y la selección de herramientas.

Las verificaciones manuales confirmaron el health check de Render, el descubrimiento de cuatro herramientas, la ejecución remota autenticada, la integración con Filesystem y Git y una conversación con contexto. El flujo Gemini hacia el servidor remoto analizó 172.16.5.130/26 y devolvió la red 172.16.5.128, el broadcast 172.16.5.191 y el rango útil 172.16.5.129 a 172.16.5.190.

10 Conclusiones

MCP permitió separar el modelo, el anfitrión y las herramientas mediante un contrato común. La implementación manual facilitó distinguir solicitudes, respuestas y notificaciones, además de comprender la función de los identificadores JSON-RPC.

El servidor local y el servidor remoto ofrecen las mismas herramientas sin cambiar su contrato. Esto demuestra que MCP permite cambiar el transporte sin modificar la lógica de negocio. El despliegue remoto agregó autenticación, TLS, health checks y validaciones propias de un servicio real.

Wireshark permitió relacionar MCP con Ethernet, IPv4, TCP, TLS y HTTP. La captura mostró la sincronización, el descubrimiento de herramientas y una ejecución exitosa. Las pruebas y las validaciones reales confirmaron que la solución cumple los objetivos del proyecto.

11 Referencias

- Model Context Protocol. Specification 2025-11-25
- JSON-RPC Working Group. JSON-RPC 2.0 Specification
- Google AI for Developers. Gemini API Function Calling
- Render. Blueprints Infrastructure as Code
- Wireshark Foundation. Wireshark User Guide